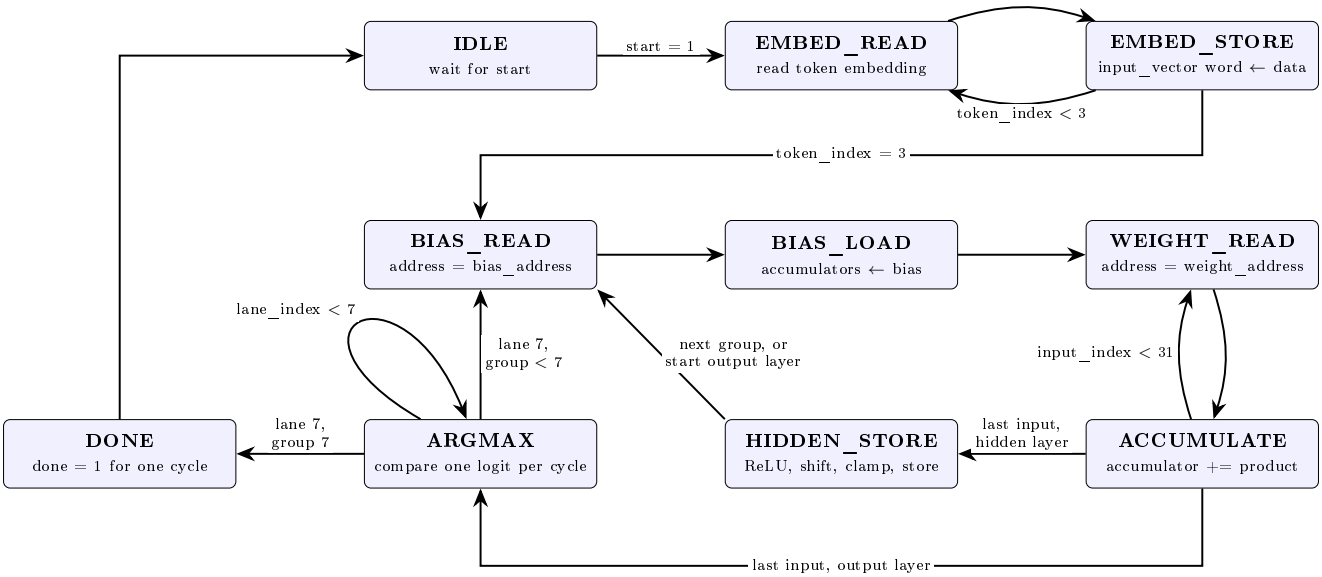
Tiny FPGA LM: inference FSM and parallel MAC lanes

Arty A7-100T, 100 MHz. One FSM sequences the embedding lookup, both fully connected layers, and the argmax. The same 8-lane MAC array and the same states serve both layers; only the `output_layer` register changes.

1. Inference FSM



State transition table

State	Condition	Next state	**Register updates at the clock edge**
IDLE	start = 0	IDLE	token_index, group_index, input_index, lane_index, weight_address, bias_address, output_layer ← 0 (every cycle spent in IDLE); argmax_clear = 1
IDLE	start = 1	EMBED_READ	same clearing; argmax_clear = 1 holds best_value at -2^{31} and best_index at 0
EMBED_READ	always	EMBED_STORE	embedding ROM latches the word at address token[token_index]
EMBED_STORE	token_index < 3	EMBED_READ	input_vector word token_index ← embedding ROM output (8 × INT8); token_index ← token_index + 1
EMBED_STORE	token_index = 3	BIAS_READ	input_vector word 3 ← embedding ROM output
BIAS_READ	always	BIAS_LOAD	bias ROM latches the word at bias_address
BIAS_LOAD	always	WEIGHT_READ	$\text{accumulator}_i \leftarrow \text{bias}_i$ for all 8 lanes; bias_address ← bias_address + 1
WEIGHT_READ	always	ACCUMULATE	weight ROM latches the word at weight_address. The activation is selected combinationally from input_vector (output_layer = 0) or hidden_buf (output_layer = 1) using input_index
ACCUMULATE	input_index < 31	WEIGHT_READ	$\text{accumulator}_i \leftarrow \text{accumulator}_i + \text{activation} \cdot \text{weight}_i$; weight_address ← weight_address + 1; input_index ← input_index + 1
ACCUMULATE	input_index = 31, output_layer = 0	HIDDEN_STORE	accumulate as above; input_index ← 0
ACCUMULATE	input_index = 31, output_layer = 1	ARGMAX	accumulate as above; input_index ← 0
HIDDEN_STORE	group_index < 3	BIAS_READ	hidden_buf byte group ← clamp(ReLU(accumulator_i) ≫ 4) for 8 lanes; group_index ← group_index + 1
HIDDEN_STORE	group_index = 3	BIAS_READ	same store; output_layer ← 1; group_index ← 0
ARGMAX	lane_index < 7	ARGMAX	if accumulator[lane_index] > best_value: best_value ← it, best_index ← {group_index, lane_index}; lane_index ← lane_index + 1
ARGMAX	lane_index = 7, group_index < 7	BIAS_READ	same compare; lane_index wraps from 7 to 0; group_index ← group_index + 1
ARGMAX	lane_index = 7, group_index = 7	DONE	same compare
DONE	always	IDLE	done = 1 this cycle; predicted_token = best_index

Registers and memory layout

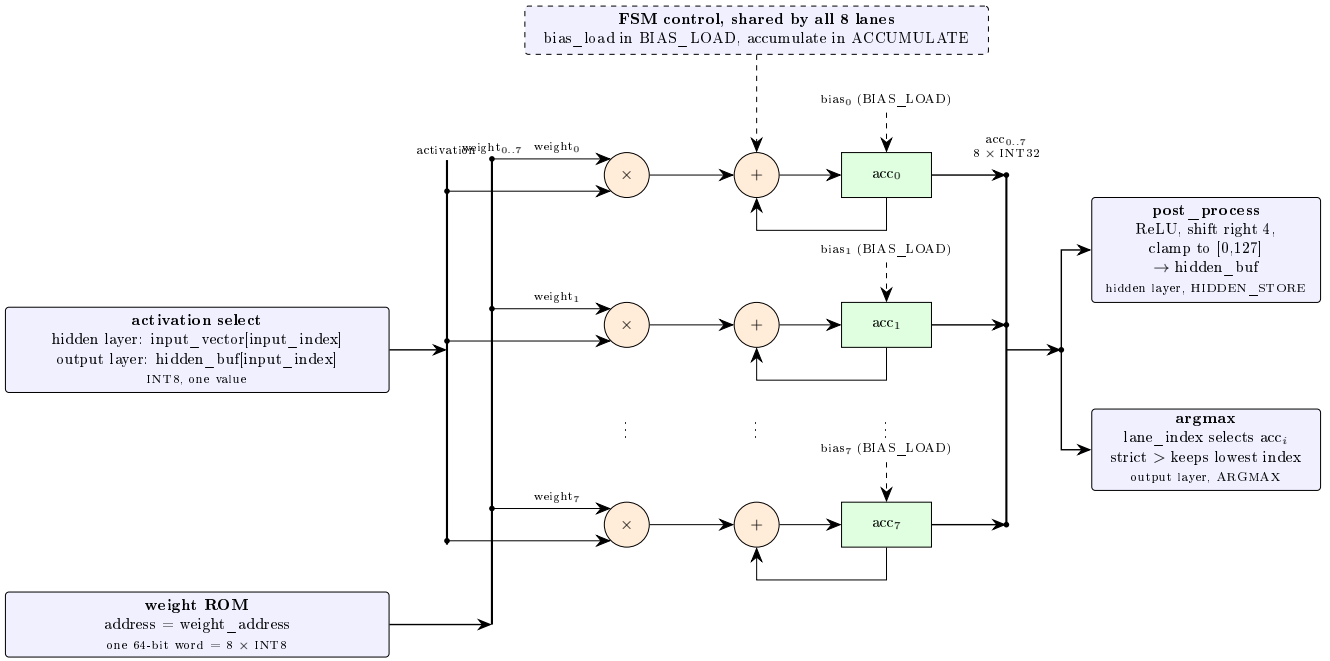
Register	Bits	Meaning
`state`	4	current FSM state
`output_layer`	1	0 = hidden layer (32→32), 1 = output layer (32→64)
`token_index`	2	which of the 4 context tokens is being embedded
`group_index`	3	which group of 8 output neurons (0–3 in the hidden layer, 0–7 in the output layer)
`input_index`	5	which of the 32 inputs within a group
`lane_index`	3	which lane's logit ARGMAX is examining
`weight_address`	9	next weight ROM word, 0 to 383; counts up and is not reset between layers
`bias_address`	4	next bias ROM word, 0 to 11
`best_value`	32 signed	largest logit so far
`best_index`	6	token ID of the largest logit so far (the output byte)

Weight ROM: 384 words of 64 bits (8 × INT8). Words 0–127 are the hidden layer (word = group·32 + input), words 128–383 are the output layer. Because the FSM visits words in order, the address is a plain counter with no multiplier. Word bits [8i+7:8i] hold $W[8 \cdot \text{group} + i][\text{input}]$ and go to lane i. The Python weight export must use this ordering.

Bias ROM: 12 words of 256 bits (8 × INT32), words 0–3 for the hidden layer and 4–11 for the output layer. Embedding ROM: 64 words of 64 bits (8 × INT8).

Latency: 8 cycles of embedding, 4 × 67 cycles for the hidden layer, 8 × 74 for the output layer, plus DONE, about 870 cycles or 8.7 μs. One UART byte at 115200 baud takes 87 μs, so the sequencing cost is negligible.

2. Eight parallel MAC lanes



Every ACCUMULATE cycle, lane i performs $acc_i \leftarrow acc_i + activation \cdot weight_i$, where the activation (INT8) is the same for all eight lanes and weight_i (INT8) is that lane's own slice of the weight word. The products are INT8 × INT8 → 16-bit signed, sign-extended into the 32-bit signed accumulator. In BIAS_LOAD the accumulator is loaded with the lane's 32-bit bias instead of the sum. The lanes never exchange data with each other, which is what distinguishes this from a systolic array.